

Setup and Review — Aurora Voice Agent

Covers objectives 2.1 (architecture), 2.2 (reservation workflow), and 2.3 (real-world call scenarios). Grounded in the actual source in this repo and in real runs captured during this review — see [stage2-evidence/](#) for raw logs and telemetry, and [DEVIATIONS.md](#) for every deliberate change made to the reference implementation during this review (persona rename, Groq removal, a Spanish-TTS-accent fix, and a new [web_demo/](#) front end), with rationale for each.

1. The mental model (what this system is)

```
caller audio -> VAD/endpointing -> STT -> AgentRouter -> LLM -> tools + RAG -> TTS -> caller
```

Four jobs, four layers:

- **Ears (STT)** — `providers.py`'s `transcribe()`. Turns audio into text. Knows nothing about hotels.
- **Brain (LLM + tool loop)** — `agent.py`'s `Agent.respond()`. Decides what to say and which tool to call. Never touches audio or the network directly — it only talks to `Provider.chat()`.
- **Hands (tools + RAG)** — `run_tool()` in `agent.py` (mock booking/availability, control actions) and `knowledge.py` (local retrieval). This is where truth lives: room inventory, policy text, confirmation IDs.
- **Voice (TTS)** — `providers.py`'s `synthesize()`, invoked from `voice_loop.py/web_demo/ server.py`'s `speak()`.

Why a cascade, not a single speech-to-speech model. Every arrow above is a real function boundary you can intercept, log, swap, or test independently:

- **Swappability:** `PROVIDER=mock` replaces the entire LLM+STT+TTS layer with deterministic, free, offline code, while `agent.py/router.py/knowledge.py` — the actual business logic — stay byte-for-byte identical. A speech-to-speech model can't be partially mocked like this.
- **Inspectability:** `telemetry.py` can time and log STT, routing, retrieval, LLM, tools, and TTS as separate spans (see §5d). A single opaque model call gives you one latency number and no way to know if the slow part was understanding the caller or picking a room.
- **Cost control:** `TTS_BACKEND=system` sends zero audio to a paid API. Text-only turns (`voice_loop.py --text`) skip STT/TTS entirely for iterating on prompt/tool logic.
- **Per-stage evals:** `run_evals.py` (see [EVAL_REPORT.md](#)) tests the LLM+tools+RAG layer in total isolation from audio, using `PROVIDER=mock`, and gets deterministic pass/fail — not possible if STT/LLM/TTS are fused into one model call.

The tradeoff, stated up front: a cascade adds latency at every hop (see §4 of `FDE-ANALYSIS.md`) and loses whatever prosody/emotion information a true speech-to-speech model could carry across the STT boundary. This repo is explicit that it's teaching the cascade, not claiming it's always the better architecture.

Workshop scope vs. production boundary, stated up front: this is a single-process, in-memory, one-call-at-a-time reference implementation. Room inventory is a hardcoded Python dict (`_ROOMS` in `agent.py`), "booking creation" doesn't persist anywhere beyond process memory, there is no authentication, and confirmation IDs are literally hardcoded (`AH-4827`, always — see §6). None of that is a bug; it's a deliberate simplification so the cascade, tool-calling, and RAG patterns are the thing being taught. `FDE-ANALYSIS.md` §7 lists what actually has to change before this could take a real call.

2. Prerequisites

- **Python:** the repo's own docs ask for 3.11/3.12. This review ran on **3.13** (the only version available on this machine) — `pip install -r pipeline/requirements.txt` succeeded cleanly, including `webtrcvad`'s native build, and all offline tests/evals passed. No known incompatibility surfaced, but this wasn't the tested-upstream version.
- **Node:** only needed for the LiveKit browser demo's `npm install` (see §4d) — not needed for any of the CLI or `web_demo/` paths.
- **Provider choice:** `PROVIDER=mock` (zero setup, deterministic, free) vs. `PROVIDER=openai` (real model, needs `OPENAI_API_KEY`, costs money). **Note:** the reference implementation originally also supported `PROVIDER=groq` as a free-tier live option sharing the same OpenAI SDK dialect as OpenAI — that support was removed from this copy at the user's explicit request partway through this review, after Groq's daily token quota was exhausted mid-session and a reproducible tool-schema bug was found and fixed (both documented in `DEVIATIONS.md` §1). OpenAI is now the only live provider in this codebase.
- **.env:** copy `pipeline/config.example.env` to `pipeline/.env`. The vars that actually matter for a text-mode review: `PROVIDER`, `OPENAI_API_KEY` (only if `PROVIDER=openai`), `TTS_BACKEND` (`system = macOS` say, zero cost; `provider = real cloud TTS`, costs money), `TELEMETRY_JSONL` (where per-turn JSONL traces get written). Confirmed gitignored both at the repo root and via a nested `Assignment_2_voice_agent/.gitignore` before ever writing a real key into it.
- **Documented-only, not run:** the LiveKit browser room demo. It needs Docker (`start_local_server.sh` runs `docker run livekit/livekit-server`), and neither Docker nor Homebrew was installed on this machine — installing them was out of scope for a code review session. **Instead, a working alternative front end (`web_demo/`) was built** — a small stdlib-only local web server with a browser chat window, real mic capture, real Whisper STT, and the identical `Agent/Provider` code the CLI uses, with TTS played through this machine's speakers exactly as `--text` mode does. See §4d and `DEVIATIONS.md` §3/§4.

- **The real-microphone CLI cascade** (`python voice_loop.py`, no `--text`) was run — live, on the user's own machine and microphone. See §4d for the transcript and a real, non-obvious latency-interpretation finding from it.

3. Folder map

Path	Purpose	Hot path?
<code>pipeline/</code>	The actual agent: prompt, tools, RAG, routing, providers, telemetry, CLI loop	Yes
<code>evals/</code>	Deterministic scenario suite (<code>core.json</code> , <code>red_team.json</code>) run against <code>agent.py</code> directly	Supporting (quality gate)
<code>livekit/</code>	Browser room demo — WebRTC transport + a REST bridge into the same agent	Alternate front end (not run — see §2)
<code>mocks/</code>	SIP/IVR concept demos (<code>demo_call.py</code> , <code>ivr_menu_mock.py</code>) — no telephony, no network	Supporting (teaching aid)
<code>knowledge/hotel_policies.md</code>	The one source-of-truth document RAG indexes	Data, hot path (read by <code>knowledge.py</code> at import time)
<code>web_demo/</code>	New this review — stdlib web server + browser chat UI, alternative to the LiveKit demo	Alternate front end (built and used)
<code>stage2-evidence/</code>	New this review — raw logs/telemetry from live-provider runs, cited throughout these docs	Evidence, not code
<code>DEVIATIONS.md</code>	New this review — every change made to the reference implementation, with rationale	Documentation

Inside `pipeline/`, in request-path order: `voice_loop.py` (loop) → `agent.py` (brain) → `router.py` + `knowledge.py` + `providers.py` (services) → `telemetry.py` (cross-cutting). `smoke_test.py`, `test_features.py`, and `scale_check.py` are offline-only, never on the runtime request path.

4. Run it manually, step by step

4a. Offline foundation

```
cd pipeline python3 -m venv .venv && source .venv/bin/activate pip install -r requirements.txt
python smoke_test.py # PASS python -m unittest -v test_features.py # 16 tests, OK
```

Actually run this session: clean install (web rtcvad's native wheel built fine on Python 3.13),
smoke_test.py → RESULT: PASS, all 16 unit tests → OK. No environment surprises.

4b. Mock text mode: the booking story, turn by turn

```
PROVIDER=mock python voice_loop.py --text
```

Turn	Caller says	Agent does	What it proves
1	"Can you tell me the weather?"	Refuses, redirects to booking	Guardrail holds before any booking context exists
2	"I need a room from Aug 12–14 for two guests."	Calls <code>check_availability</code> , lists Standard Queen at \$189/night	Tool call, not model memory
3	"Yes, book it for Priya Shah at priya@example.com."	Calls <code>create_booking</code> , returns confirmation AH-4827	Confirmation ID comes from the tool, not the model
4	"Actually, connect me to a person"	<code>transfer_to_human</code> , <code>[action: transfer]</code>	Control action distinct from a text reply
5	"Goodbye"	<code>end_call</code> , <code>[action: hangup]</code>	Natural call end

This is smoke_test.py's exact script — deterministic because MockProvider is rule-based, not a real model.

4c. Live text mode (OpenAI): what changes

```
PROVIDER=openai python voice_loop.py --text
```

Everything downstream of provider.chat() is unchanged code — same Agent, same TOOLS, same run_tool(). What changes is real, nondeterministic model output and real latency. Concretely, from this review's actual live runs (stage2-evidence/):

- The confirmation ID is **still** always AH-4827 — it's hardcoded in `run_tool()`, not generated, live model or not (see §6).
- Given 5 equally-valid rooms, a live model asked for clarification on "book it" instead of guessing — mock mode never exercises this because `MockProvider` always offers exactly one room.
- Turn latency became real and highly variable: 663ms–3654ms LLM, 5–24 **seconds** TTS (spoken audio via `say`, not network latency) per turn. See `stage2-evidence/README.md` for the full breakdown — this is the raw data `FDE-ANALYSIS.md` §4 builds on.
- One genuinely reproducible bug was found and fixed here: the live model emitted the `guests` tool argument as a quoted string, which the tool schema declared as `integer`, causing every booking attempt to hard-crash with an uncaught `openai.BadRequestError` (no `try/except` anywhere in `agent.py/voice_loop.py`). Fixed by declaring `guests` as `"type": "string"` in `TOOLS` (matching observed reality) plus a defensive `_coerce_guests()` in `run_tool()`. Full detail in `DEVIATIONS.md`.

4d. Documented-only vs. built-and-used vs. actually run live

The **real-mic CLI cascade** (`python voice_loop.py`) — `record_utterance()` uses `sounddevice+webrtcvad` to capture until `ENDPOINT_SILENCE_MS` (default 600ms) of trailing silence is detected, then transcribes the buffered PCM — **was run live**, by the user, on their own microphone (`stage2-evidence/04_real_mic.log`). Three real turns: "I'd like to book a room" (correctly asks for the missing check-in/check-out/guest-count slots rather than guessing) → "Could you speak in Spanish?" (a genuine live mid-flow language switch — the clarifying question gets correctly re-asked in Spanish, and the partially-filled booking-flow state survives the switch) → "Stop, end call." (hangup, with the reply correctly still in Spanish, proving language state persists all the way to call end).

This produced the only `capture/stt` timings in this entire review (`--text` mode skips both by construction) and surfaced a real interpretation trap worth internalizing before reading any latency number in this repo: `capture` was 9-11 **seconds** per turn, but that is almost entirely "however long the human took to speak," not system latency — the VAD timer starts the instant recording begins. And `tts` (10.4s on turn 1) is *time to finish speaking the whole reply*, not *time to first audio* — `say` starts producing sound almost immediately, but `subprocess.run()` only returns once the entire utterance is done, and the telemetry schema has no field to capture the difference. The caller's actual perceived "how long until I hear anything" on turn 1 is closer to `capture + stt + routing + llm` ≈ **14 seconds** — still slow, but a different number and a different bottleneck than a naive read of the `tts` field would suggest. Full analysis in `stage2-evidence/README.md` Finding 3; this is load-bearing for `FDE-ANALYSIS.md` §4.

The **LiveKit browser room demo** needs `docker run livekit/livekit-server` (via `start_local_server.sh`) plus `npm install` for the browser client — neither Docker nor Homebrew was available on this machine, and installing a system package manager plus a container runtime was judged out of scope for what should be a code review. Documented from `livekit/README.md` and

`livekit/talk_server.py`'s source (see §6) rather than run live.

In its place, `web_demo/` was built and actually run end-to-end: a ~200-line `stdlib` `http.server` (no new pip dependencies) serving a single HTML page with a "Start recording" button. Browser `MediaRecorder` captures a clip → `POST /turn` → the server transcribes it via `Whisper`, runs it through the *exact same* `Agent/Provider` objects `voice_loop.py` `--text` uses, and speaks the reply through `say/afplay` on this machine (browser and server share the same speakers in this demo, so no audio needs to be streamed back over HTTP). This was verified working live against OpenAI, including recovering the fix in §4c. Two real bugs were found and fixed while building it — both documented in full in `DEVIATIONS.md` §4, and worth reading for objective 2.4: one was a classic "don't respond before draining the request body on a keep-alive connection" bug that corrupted subsequent requests; the other was a client/server call-state desync (page reload resets client state but not server state) surfaced as a confusing 409.

5. How to test and review it manually

5a. The reservation workflow end to end (objective 2.2)

Availability → booking → confirmation, multi-turn, is `check_availability` then `create_booking` in `agent.py`'s `TOOLS`, executed by `run_tool()` (lines ~270–300). A tool call in the trace looks like this (from `stage2-evidence/02_booking_completion_telemetry.jsonl`, redacted fields as `telemetry.py`'s default):

```
{"name": "tool.requested", "attributes": {"tool": "create_booking", "arguments": "[OMITTED:...]"}} {"name": "tool.result", "attributes": {"tool": "create_booking", "result": "[OMITTED:...]", "action": null}}
```

Multi-turn state lives entirely in `Agent.messages` (the running chat history) plus three explicit fields: `current_language`, `current_locale`, and `router` (an `AgentRouter` instance). There is no external session store — one `Agent` instance is one call, full stop. The real captured booking (`stage2-evidence/02_booking_completion.log`): caller specifies a Standard Queen explicitly, agent confirms rate, caller confirms with name+email, agent returns **"Confirmation AH-4827"** — the same ID every single time, because it's a string literal in `run_tool()`, not generated. This is fine for a teaching demo and a real gap for production (§7 of `FDE-ANALYSIS.md`: no persistence, no idempotency, no real ID generation).

5b. Real-world call scenarios (objective 2.3)

- **Off-topic redirect:** `required_tool_for()` in `agent.py` isn't consulted here — off-topic refusal is left to the system prompt's guardrail instructions, and it holds reliably across every run in this review (weather, FIFA scores, etc. all correctly redirected).

- **Transfer to front desk:** `transfer_to_human` sets `action: "transfer"`, which `voice_loop.py` maps to `[transferring to front desk: SIP REFER to front-desk]`. **Important finding from this review:** repeated live testing (5 attempts, see `stage2-evidence/README.md` Finding 1) showed the live model sometimes chains `transfer_to_human()` immediately followed by `end_call()` in the *same turn* — and because `agent.py`'s tool loop tracks "the last action seen" with no validation that transfer and hangup are mutually exclusive, the hangup silently overwrites the transfer. **2 of 5 attempts** in this review's sample produced this: the caller is told "I'm transferring you" but the call actually ends. Mock mode structurally cannot catch this — `MockProvider.chat()` only ever returns one tool call per turn, never a chain. This is real, load-bearing evidence for `FDE-ANALYSIS.md` §2 and `EVAL_REPORT.md` §5–6.
- **Natural hangup:** `end_call` → `action: "hangup"` → `[call ended: SIP BYE]`. Reliable across every test.
- **The fallback ladder when the model misfires:** there mostly isn't one. `voice_loop.py` has zero try/except around `agent.respond()` — a live provider error (the guests-string bug, a rate limit, a network blip) crashes the entire process with a raw Python traceback, ending the call with no SIP-mappable signal at all. `livekit/talk_server.py`, by contrast, wraps every request handler in try/except and returns a JSON error without killing the server — a real, notable inconsistency in error-handling maturity between the two front ends in the same codebase. See §6 and `FDE-ANALYSIS.md` §3.

5c. Language switch (EN ↔ ES) and why it's a validated control tool

`set_language` is a structured tool, not free-text detection. `explicit_language_request()` in `agent.py` requires the caller's utterance to literally name the target language before `AgentRouter.set_language()` is allowed to change state — this is why "¡Gracias!" correctly does **not** flip the session back to Spanish (verified live, `stage2-evidence/01_full_sequence.log` turn 9→10: switches to English on an explicit request, then a bare "¡Gracias!" leaves it in English). This session also found and fixed a real gap: `agent.current_locale` was tracked but never actually passed to `provider.synthesize()`, so Spanish replies were spoken in whatever the OS default voice happened to be — audibly non-native. Fixed by threading `locale` through to TTS and mapping `es-ES` → the native macOS voice `Mónica` (exact accented name — `say -v <bad-name>` silently falls back to default rather than erroring, which is itself worth noting as a silent failure mode; see `DEVIATIONS.md` §3).

5d. Reading telemetry: one turn's JSONL line, field by field

From a real captured line (`logs/voice-events.jsonl`, sensitive fields redacted by `telemetry.py`'s `_sanitize()`):

```
{ "schemaVersion": "1.0", "traceId": "...", // one turn, globally unique "sessionId":  
"cli-...", // one call "turnId": "turn-...", "totalMs": 18998.0, "timings": {"routing": 0.0,  
"retrieval": 0.6, "llm": 1652.0, "tools": 0.6, "tts": 17345.0}, "attributes": {"language":  
"en", "sources": ["hotel_policies.md#Cancellation"], "action": null}, "events": [ /* ordered:  
routing.started -> ... -> tool.requested -> tool.result -> ... */ ] }
```

`TELEMETRY_INCLUDE_CONTENT=false` (the default) replaces actual transcript/reply text with `[OMITTED:<length>]`; sensitive tool fields (`guest_name`, `contact`, `email`, `phone`) are `[REDACTED]` regardless of that setting. Every stage of the cascade gets its own timing — this is what makes the per-stage latency table in §4c and `FDE-ANALYSIS.md` §4 possible at all.

5e. Running the evals

See `EVAL_REPORT.md` for the full case-by-case breakdown. Headline: 14/14 scenarios pass (`python run_evals.py --suite all, PROVIDER=mock, deterministic`) — 12/12 was the upstream baseline as cloned; 14/14 reflects the 2 cases added during this review's Stage 4 (§7 of `EVAL_REPORT.md`).

6. How to review the code (request-path order)

providers.py — one adaptor (OpenAI SDK; originally two, see `DEVIATIONS.md` §1), three methods (`chat/transcribe/synthesize`), plus a full `MockProvider` implementing the identical interface. The key it reads is chosen entirely by `PRESETS[name]["api_key_env"]` — never hardcoded, never logged.

Design decision worth noting: `MockProvider.chat()` is a hand-written rule engine (keyword matching, not an LLM) — this is why it's deterministic and why it structurally cannot reproduce live-model failure modes like the compound-tool-call bug in §5b.

Failure mode: `Provider.synthesize()`'s system backend (`subprocess.run(["say", ...])`) has no error checking at all — an invalid voice name, per this review's own experience, doesn't raise, it silently plays the wrong voice.

agent.py — the actual brain: `SYSTEM_PROMPT`, `TOOLS` schema, the tool-calling `while` loop in `Agent.respond()`, and `run_tool()`'s mock implementations.

Design decision worth noting: `required_tool_for()` force-routes high-confidence policy/amenity phrases to `search_hotel_knowledge` **before** the first model call, specifically so a prior off-topic refusal can't suppress grounding on the very next in-scope question (this is exactly what `grounding.after_off_topic` in the eval suite checks).

Failure mode: the tool loop's `action` variable is simply "the last control signal seen" with no validation — see §5b's transfer/hangup finding. This is the single most important correctness gap found in this entire review.

knowledge.py — an in-memory SQLite FTS5 index over `knowledge/hotel_policies.md`, chunked by Markdown `##` headings, with a hand-rolled EN/ES query-expansion dict (`_QUERY_EXPANSIONS`) and a non-FTS lexical fallback if SQLite's FTS5 extension is unavailable.

Design decision worth noting: query expansion happens at query time (e.g. "mascota" also searches "pets"/"dogs"), not by duplicating the source document per language — one Markdown file serves both languages.

Failure mode: `search()` has **no relevance-score threshold** — it always returns up to `limit` (default 3) results regardless of match quality. This is the likely explanation for a real observed case (`stage2-evidence/01_full_sequence`, turn 11) where a check-in-time question also pulled in an unrelated Cancellation-policy source.

router.py — small and clean: a frozen `Route` dataclass, an `AgentRouter` holding one mutable `language` field, and `explicit_language_request()`'s token-based guard (see §5c).

Design decision worth noting: the guard requires the *language name itself* as a token match ("english"/"spanish"/"ingles"/"español"), not an LLM judgment call — a small, fast, deterministic check that fully explains the "¡Gracias!" behavior in §5c without needing another model call.

Failure mode: none observed — this module's scope is small enough that it's hard to get wrong, and it did not need touching or fixing anywhere in this review.

telemetry.py — `TurnTrace` (events + span timings + attributes), `_sanitize()` (redaction), `write_trace()/format_trace()` (JSONL + terminal output).

Design decision worth noting: redaction is **key-name-based** (`_SENSITIVE_KEYS`, `_CONTENT_KEYS`), applied recursively to nested dicts/lists — so any new tool argument named `contact` or `email` gets redacted automatically with zero code changes elsewhere.

Failure mode: redaction is a fixed keyword list — a tool argument holding sensitive data under an unlisted key name (e.g. a future `notes` field containing a phone number) would not be caught.

voice_loop.py — the cascade wiring for both CLI modes (`--text` and real mic), per-stage `trace.span()` timing, and the endpointing logic (`record_utterance()`'s VAD loop).

Design decision worth noting: `--text` mode and mic mode share the exact same downstream code from `agent.respond()` onward — text mode isn't a separate stub, it's the same loop with STT skipped, which is precisely why mock-text-mode testing is representative of the real agent logic.

Failure mode: **zero try/except around `provider.chat()` or `agent.respond()`** — see §5b. This is the single most consequential operational gap in the whole reference implementation, and it's present in every CLI-based path (`voice_loop.py`, `smoke_test.py` is insulated only because it uses the exception-free mock).

livekit/talk_server.py — an HTTP bridge (`SimpleHTTPRequestHandler`), *not* a LiveKit room-native agent worker. Confirmed directly from source: caller/agent audio flows through plain `POST /voice-agent` and `POST /agent` request/response cycles, with LiveKit used only for the room/participant/token layer (`_token()`, `/token`, `/state`) — the actual STT→agent→TTS pipeline never subscribes to a published LiveKit audio track. This matches the README's own stated

"LiveKit Boundary" caveat, now verified in code rather than taken on faith.

Design decision worth noting: session state is a process-global dict (`_agent_sessions`) keyed by a client-supplied `x-session-id` header, with a barge-in echo-suppression heuristic (`_is_probable_playback_echo()`) that filters out short acknowledgment phrases ("thanks", "you're welcome") likely picked up from the agent's own TTS bleeding into the mic.

Failure mode, and a real inconsistency worth flagging: every handler here **is** wrapped in try/except, returning a JSON `{"error": ...}` with a 500 instead of crashing — meaning this front end is strictly more resilient to a provider failure than `voice_loop.py`'s CLI path, in the same codebase, for the same underlying bug. Also: `x-session-id` has no ownership check at all — any client supplying a guessed session ID can resume another session's conversation state.

7. Common gotchas

Symptom	Cause	Fix
<code>ValueError: Unknown PROVIDER 'groq'</code>	This copy of the repo has Groq removed (see <code>DEVIATIONS.md</code>)	Use <code>PROVIDER=mock</code> or <code>PROVIDER=openai</code>
Booking crashes with <code>openai.BadRequestError / tool_use_failed</code>	A live model emits <code>guests</code> as a string against an <code>integer</code> -typed schema	Already fixed in this repo (<code>TOOLS</code> declares <code>guests</code> as <code>string + _coerce_guests()</code>) — if you see this again, the schema/model pairing regressed
Whole process dies with a raw traceback mid-call	No try/except around <code>provider.chat()/agent.respond()</code> in <code>voice_loop.py</code>	Known gap, not yet fixed — see <code>FDE-ANALYSIS.md §3</code>
Caller told "transferring" but call just ends	Live model chains <code>transfer_to_human + end_call</code> in one turn; last-action-wins silently drops the transfer	Known gap, not yet fixed (~40% reproduction rate in this review's sample) — see §5b
Spanish TTS sounds non-native	<code>system</code> TTS backend ignores <code>locale</code> unless explicitly wired (was the case before this review's fix)	Fixed in this repo; if regressed, check <code>_system_tts_args()</code> in <code>providers.py</code> uses the <i>exact</i> accented voice name (say <code>-v <bad-name></code> fails silently, not loudly)
<code>webrtcvad</code> fails to build	Missing <code>PortAudio</code> , or <code>setuptools/pkg_resources</code> mismatch	<code>brew install portaudio</code> ; <code>requirements.txt</code> already pins <code>setuptools<81</code> for this
LiveKit demo won't start	No Docker daemon running	Install Docker (or use <code>web_demo/</code> as a working alternative — see §4d)

Web demo POST /turn returns 409 repeatedly	Server-side call already ended (hangup/transfer) but the page was reloaded, resetting only client-side state	Click "New call" (calls /reset) instead of reloading the page
Same eval suite, mock always green, live sometimes red	Mock is a deterministic rule engine; live models are not	Expected — see EVAL_REPORT.md §5 for exactly what this does and doesn't prove

8. 10-minute self-check

- [] `python smoke_test.py` → `RESULT: PASS`
- [] `python -m unittest -v test_features.py` → 16 tests, `OK`
- [] `PROVIDER=mock python voice_loop.py --text` → full booking story completes with confirmation `AH-4827`, then transfer, then hangup
- [] `.env` confirmed gitignored before adding any real key (`git check-ignore -v pipeline/.env`)
- [] `PROVIDER=openai python voice_loop.py --text` → one live turn succeeds (confirms the `guests-type` fix holds)
- [] Ask "What is the weather?" → redirected to hotel reservations, not answered
- [] Ask a cancellation/pet/check-in policy question → reply cites a `hotel_policies.md#...` source in the trace, not just fluent text
- [] "Please speak Spanish" → language flips to `es`; a bare "¡Gracias!" afterward does **not** flip it back
- [] `tail -n 1 logs/voice-events.jsonl | python3 -m json.tool` → shows per-stage timings and redacted sensitive fields
- [] `cd evals && python run_evals.py --suite all` → `Score: 14/14 scenarios passed`